Welcome to your next lesson: String skills upgrade i: indexing and slicing.

I'm preparing your personalized lesson based on what you've learned, what's coming next, and the best way to help you practice.

Step by step, you're moving forward.

Let’s start with something simple and concrete, Ron 🎯

Imagine this string in Python:

Python

```
word = "Python"
```

Strings are like a row of boxes, one character per box. In Python, we count positions starting at 0, not 1:

```
word[0] is 'P'
word[1] is 'y'
word[2] is 't'
```

…and so on

So:

Index = position number of a character in the string (starting at 0).

Quick check: does the idea “first character is index 0, second is index 1, …” feel clear, or a bit weird/confusing?

That makes sense

What do you think the answer is? Take your time with the question below 👇

Given this code:

```
word = "Python"
print(word[3])
```

What character will be printed?

A.

'h'

B.

't'

C.

'o'

D.

'n'

Nice work, Ron 🎉

You were right: word[3] prints 'h'.

Step by step:

word = "Python"
Index positions are:
word[0] → 'P'
word[1] → 'y'
word[2] → 't'
word[3] → 'h'

So index 3 lands on the 4th character, which is 'h'.

Next tiny step: instead of a single character, we can grab parts of the string using slicing, like word[1:4].

Before I explain slicing, how comfortable do you feel on 0-based indexing right now?

Very comfortable
Mostly okay but want one more example
Still fuzzy
Very comfortable

Awesome 😄 That's exactly where I want you before we add the next skill.

Now let's level up to slicing: taking a chunk of a string.

Using the same string:

Python

word = "Python"
If we write:

Python

print(word[1:4])
this means:

start at index 1 ('y')
go up to but not including index 4
so it takes indices 1, 2, 3 → 'y', 't', 'h' → prints "yth"

So:

word[start:stop] → takes characters from start index up to, but not including stop.

Does that "start included, stop excluded" idea make sense so far, or should I show one more quick example before quizzing you?

Makes sense to me

What do you think the output is? Pick an option below 👇

Given this code:

word = "Python"
print(word[1:4])
What will be printed?

A.

"Pyt"

B.

"yth"

C.

"ytho"

D.

"Pyth"

Good try, Ron — this is exactly the tiny detail that trips people up at first 👌

With:

Python

```
word = "Python"
print(word[1:4])
```

Index map:

```
word[0] → P
word[1] → y
word[2] → t
word[3] → h
word[4] → o
word[5] → n
```

[1:4] means:

start at index 1 → y
go up to but NOT including index 4

So it takes indices 1, 2, 3 → y, t, h → "yth" ☑

"Pyt" would be indices 0,1,2 ([0:3]), which is why that one was off.

Want to try one more slice like this, or would you rather move on to the next idea: adding a step like [start:stop:step]?

I'm ready for the next idea

Great, let's add step to slicing 🚶‍♂️

We still use word = "Python", but now:

Python

```
print(word[0:6:2])
```

Slice [start:stop:step] means:

start at index 0 → P
go up to but not including index 6 (the end)
move in jumps of 2 (step = 2)

So it takes indices:

0 → P
2 → t
4 → o

Result: "Pto"

So:

word[start:stop:step] → take characters from start to just before stop, jumping step each time.

Does that rule — “jump by step through the indices” — feel clear?

Yes, that's clear

What do you think the output is? Choose an option below 👇

Given:

word = "Python"
print(word[0:6:2])
What will be printed?

A.

"Pyt"

B.

"Pto"

C.

"Phn"

D.

"Pyon"

Nice job, Ron — you nailed that 👏

For:

Python

word = "Python"
print(word[0:6:2])
we have:

indices: P(0) y(1) t(2) h(3) o(4) n(5)
[0:6:2] means:
start at 0 → P
then 2 → t
then 4 → o

So it collects "Pto" ☑

Next concept is negative indices (counting from the end of the string). Want to go there now?

Yes, let's learn about negatives

Awesome, let’s flip the string around a bit 😄

Take this again:

Python

word = "Python"
With negative indices, Python counts from the right:

word[-1] → last character → 'n'
word[-2] → second from last → 'o'
word[-3] → 'h'
…and so on

So you can think:

Rightmost char is -1, then you move left: -2, -3, etc.

Quick check in your own words (no need to be fancy): what do you expect word[-1] and word[-2] to be?

n and o

Perfect ☑ Exactly right:

word[-1] → 'n'
word[-2] → 'o'

You’re thinking about the string from the right side, which is the key idea.

Next tiny step is to combine what you know and check it with a quick question using negatives (no trick).

Ready for a short multiple□choice check on negative indices?

Yes, I'm ready

What do you think? Pick an answer below 👇

Given:

word = "Python"
print(word[-1])
What will be printed?

A.

"P"

B.

"n"

C.

"h"

D.

"o"

Nicely done, Ron 🎉

For:

Python

```
word = "Python"
print(word[-1])
```

Python counts from the right when the index is negative:

word[-1] → last character → 'n' ☑

So your answer is spot on.

Last key idea for this lesson is using negative indices in slices, like word[-3:] or even reversing with word[::-1].

Do you want to see how negative indices work with slices next?

Yes, show me examples

Great, let’s keep it concrete and visual 🔍

Still using:

Python

```
word = "Python"
# indexes:  P   y   t   h   o   n
#           0   1   2   3   4   5
#          -6  -5  -4  -3  -2  -1
```

A few negative-index slices:

word[-3:]

start at index -3 → 'h'
go to the end
result: "hon"
word[:-2]

start at the beginning (since start is empty)
stop at index -2 (but don’t include it) → stop before 'o'
result: "Pyth"
word[::-1]

start/end left empty
step is -1 → walk backwards one step at a time
result: "nohtyP" (the whole string reversed)
Does at least one of these feel clear enough that you could predict its output if I changed the word, or do you want to zoom in on one example (like [-3:] or [::-1]) first?

I'm ready to try predicting

What do you think the output is here? Take your time and pick an option below 👇

Given:

```
word = "Python"
print(word[-3:])
```

What will be printed?

A.

"hon"

B.

"tho"

C.

"yth"

D.

"ton"

Nice, Ron — you're really getting this 🎯

For:

Python

```
word = "Python"
print(word[-3:])
```

we use the index map:

```
P  y  t  h  o  n
0  1  2  3  4  5
-6 -5 -4 -3 -2 -1
```

[-3:] means:

start at -3 → 'h'
go to the end of the string

So it takes 'h', 'o', 'n' → "hon" ☑

You've now used:

0-based indexing (word[3])
basic slices ([start:stop])
steps ([start:stop:step])
negative indices and negative□based slices

One last fun challenge and then we can wrap this lesson 🎓

Think about the string "PYTHON".

Is there any slice PYTHON[? : ? : ?] that gives "NYT" in that order?

In other words: "NYT" is:

not in order in the original string, and
slices must move in a steady step (forward or backward)

So the correct answer is: no valid slice can produce "NYT".

That's because:

forward: letters appear as P Y T H O N
backward: N O H T Y P
there's no way to "jump around" to reorder as N Y T using a simple slice.
You've hit all the mastery goals for this lesson 💪

Are you ready to wrap up this lesson and move on to the next one, or do you want one

more quick practice with a custom string (like your name or an email) first?

python (-1, 2:1) is al i can think but not sure of the step 1 fro the move to 2 i think answers o

I like how you're thinking it through, Ron — that's exactly the right kind of reasoning 🔍

Let me gently untangle a couple of things you mixed together:

word[-1] by itself is just indexing, not slicing.
A proper slice has colons, like: word[start:stop:step].

So something like:

Python

"PYTHON"[-1:2:1]
means:

start at index -1 → 'N'
go forward to index 2 (but not including 2)
with step 1 (normal forward direction)

But here's the key rule:

If step is positive, start must be before stop going left→right, or else the slice is just empty.

Index map:

P(0) Y(1) T(2) H(3) O(4) N(5)
P(-6) Y(-5) T(-4) H(-3) O(-2) N(-1)

-1 (5) is actually after 2, so with step 1 this slice gives "" (empty string), not "NYT".

And even if we fix direction, there's still no way to hit 'N', then 'Y', then 'T' in order with a constant step — that's why the correct idea is: no slice can produce "NYT".

Want to try writing one valid slice on "PYTHON" that you know the result of (like "PYT" or "NO"), and tell me what you expect it to return?

python((0:2) == "pyt"

Love that you're trying to write it in Python form, Ron 🚲

Let's slow it down and clean up the syntax step by step:

You wrote something like:

Python

python((0:2) == "pyt"
To compare a slice of the string "python" with "pyt", the proper Python code would be:

Python

word = "python"

print(word[0:3] == "pyt")
Two key fixes:

You need a variable name (like word), not the word python as code.
For "pyt", the slice must be [0:3] (indices 0,1,2) because the stop is not included.

So word[0:2] would give "py", but word[0:3] gives "pyt" ☑

Can you tell me (just in words here):

What characters do you think word[1:4] would give if word = "python"?
wasnt runing ode justprograming huor it eant my aswer wa gonna have the value but okay

Gotcha, that makes sense 👍

You were kind of “thinking in code” to show the idea, not trying to run it. That’s a good habit.

And your idea was on track: you wanted a slice of "python" that equals "pyt".

The only detail to tune is the stop index:

"python" indices:
p(0) y(1) t(2) h(3) o(4) n(5)
To get "pyt" you need indices 0, 1, 2
So the slice is word[0:3], not word[0:2]

Because stop is not included.

Let’s do one more quick prediction to lock this in:

If:

Python

word = "python"
what characters do you think word[1:4] gives?

You can just answer something like "yth" style, no need to write code.

yth

Exactly right — word[1:4] is "yth" ☑

Step by step for "python":

p(0) y(1) t(2) h(3) o(4) n(5)
[1:4] → indices 1, 2, 3 → y, t, h → "yth"

You’ve now:

used 0-based indexing
sliced with [start:stop] and [start:stop:step]
used negative indices and negative-based slices

You’ve hit all the goals for this lesson 🎓

Want to wrap up this lesson and move to the next one, or do one more challenge-style slice first (your choice)?

I'm ready to move on

Nice work, Ron 🎯

You've mastered:

0-based indexing – word[3] → single characters
Basic slices – word[start:stop] with stop not included
Stepped slices – word[start:stop:step] like word[0:6:2]
Negative indices – word[-1], word[-3:], and how they count from the end
You're thinking in code and reasoning through examples, which is exactly what will make the next topics feel easier.

When you're ready, go ahead and start the next lesson in the Learn tab, and I'll keep walking through it with you step by step 🚀

Thank you for your feedback!